Week 12

Cases of RL & Towards algorithmic optimization

Nebius Academy

PPO, GRPO

PPO

$$\mathcal{L} = \mathbb{E}_{x\sim\mathcal{D}}\,\mathbb{E}_{\mathbf{y_{\leqslant t}}\sim\pi_{\mathbf{old}}(\mathbf{y}|\mathbf{x})}\min\left[\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\text{old}}(y_t|x,y_{<t})}\widehat{A}_t(x,y_{<t},y_t), \text{clip}\left(\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\text{old}}(y_t|x,y_{<t})}, 1-\varepsilon, 1+\varepsilon\right)\widehat{A}_t(x,y_{<t},y_t)\right] \approx$$

$$\approx \frac{1}{|B|}\sum_{x\in B, y\sim\pi_{old}(y|x)}\frac{1}{\text{len}(y)}\sum_{t=1}^{\text{len}(y)}\min\left[\ldots\right]$$

The RL objective

$$\mathcal{L}_{RL} = \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi_\theta(y|x)} R(x, y)$$

where

- $\mathcal{D}$ is the dataset of training prompts
- $\pi_\theta(y|x)$ is the LLM (policy); x is a prompt; y is the completion
- $R(x, y)$ is the reward

The RL objective

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(y|x)}R(x,y)$$

where

- $\mathcal{D}$ is the dataset of training prompts
- $\pi_\theta(y|x)$ is the LLM (policy); x is a prompt; y is the completion
- $R(x,y)$ is the reward

$$\mathbb{E}_{y\sim\pi_\theta(y|x)}R(x,y) = \sum_{y\sim\pi_\theta(y|x)} \pi_\theta(y|x)\cdot R(x,y)$$

The quest for reducing variance

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(y|x)}R(x,y)$$

where

- $\mathcal{D}$ is the dataset of training prompts
- $\pi_\theta(y|x)$ is the LLM (policy); x is a prompt; y is the completion
- $R(x,y)$ is the reward

We want to try to reduce the variance of the gradients and to make the training process a little less noisy

Step 1 – advantage

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(y|x)}\left(R(x,y) - \widehat{V}(x)\right)$$

where $\hat{V}(x)$ is a **trained estimate** of the value function (expected reward)

$\hat{A}(x,y) = R(x,y) - \widehat{V}(x)$ is called **advantage**

You can prove: subtracting $\hat{V}(x)$ doesn't change the gradient

Step 1 – advantage

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(y|x)}\widehat{\boldsymbol{A}}(\boldsymbol{x},\boldsymbol{y})$$

where $\hat{V}(x)$ is a **trained estimate** of the value function (expected reward)

$\hat{A}(x,y) = R(x,y) - \widehat{V}(x)$ is called **advantage**

You can prove: subtracting $\hat{V}(x)$ doesn't change the gradient

Step 2 – importance sampling

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(y|x)}\hat{A}(x,y)$$

The problem: we don't generate things on the fly; our actual loss is

$$\mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)}\hat{A}(x,y) =$$

$$= \mathbb{E}_{x\sim\mathcal{D}} \sum_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)} \pi_\theta(y|x)\hat{A}(x,y)$$

Step 2 – importance sampling

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(y|x)}\hat{A}(x,y)$$

The problem: we don't generate things on the fly; our actual loss is

$$\mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)}\hat{A}(x,y) =$$

$$= \mathbb{E}_{x\sim\mathcal{D}} \sum_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)} \pi_\theta(y|x)\hat{A}(x,y) =$$

$$= \mathbb{E}_{x\sim\mathcal{D}} \sum_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)} \frac{\boldsymbol{\pi_\theta^{old}}(y|x)}{\boldsymbol{\pi_\theta^{old}}(y|x)}\pi_\theta(y|x)\hat{A}(x,y) = \mathbb{E}_{x\sim\mathcal{D}} \sum_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)} \boldsymbol{\pi_\theta^{old}}(y|x)\frac{\pi_\theta(y|x)}{\boldsymbol{\pi_\theta^{old}}(y|x)}\hat{A}(x,y)$$

Step 2 – importance sampling

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)}\frac{\boldsymbol{\pi_\theta(y|x)}}{\boldsymbol{\pi_\theta^{old}(y|x)}}\hat{A}(x,y)$$

The problem: we don't generate things on the fly; our actual loss is

$$\mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)}\hat{A}(x,y) =$$

$$= \mathbb{E}_{x\sim\mathcal{D}}\sum_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)}\pi_\theta(y|x)\hat{A}(x,y) =$$

$$= \mathbb{E}_{x\sim\mathcal{D}}\sum_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)}\frac{\boldsymbol{\pi_\theta^{old}}(y|x)}{\boldsymbol{\pi_\theta^{old}}(y|x)}\pi_\theta(y|x)\hat{A}(x,y) = \mathbb{E}_{x\sim\mathcal{D}}\sum_{y\sim\boldsymbol{\pi_\theta^{old}}(y|x)}\boldsymbol{\pi_\theta^{old}}(y|x)\frac{\pi_\theta(y|x)}{\boldsymbol{\pi_\theta^{old}}(y|x)}\hat{A}(x,y)$$

Step 3 – anti-reward hacking

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)}\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y)$$

Reward hacking: the model learns to maximise the reward at the expense of text generation proficiency (or whatever else we wanted from it)

Step 3 – anti-reward hacking

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)}\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y)$$

Reward hacking: the model learns to maximise the reward at the expense of text generation proficiency (or whatever else we wanted from it)

Popular remedy: forbid the model from straying far away from the pre-RL state:

$$\begin{cases} \mathcal{L}_{RL} \to \max\limits_{\theta} \\ KL\left(\pi_\theta \parallel \pi_{ref}\right) \leq \delta \end{cases}$$

where $KL\left(\pi_\theta \parallel \pi_{ref}\right) = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta(y|x)}\frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$

Step 3 – anti-reward hacking

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)}\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y) - \boldsymbol{\beta \cdot KL(\pi_\theta \parallel \pi_{ref})}$$

Reward hacking: the model learns to maximise the reward at the expense of text generation proficiency (or whatever else we wanted from it)

Popular remedy: forbid the model from straying far away from the pre-RL state:

$$\begin{cases} \mathcal{L}_{RL} \to \max\limits_{\theta} \\ KL(\pi_\theta \parallel \pi_{ref}) \leq \delta \end{cases}$$

Step 3 – anti-reward hacking

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)}\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y) - \boldsymbol{\beta \cdot KL\left(\pi_\theta \,\|\, \pi_{ref}\right)} =$$

$$= \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)}\left[\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y) - \boldsymbol{\beta \cdot \frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}}\right]$$

Step 4 – Clipped Surrogate Objective

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)}\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y)$$

The idea is to increase the probability of actions with $\hat{A} > 0$ and decrease the probabilities of actions with $\hat{A} < 0$.

But we don't want the changes $\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}$ in these probabilities to be too drastic.

Let's make large values of $\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}$ useless for the training to discourage from drastic changes.

Step 4 – Clipped Surrogate Objective

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)}\boldsymbol{clip}\left(\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}, \boldsymbol{1-\varepsilon, 1+\varepsilon}\right)\hat{A}(x,y)$$

The idea is to increase the probability of actions with $\hat{A} > 0$ and decrease the probabilities of actions with $\hat{A} < 0$.

But we don't want the changes $\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}$ in these probabilities to be too drastic.

Let's make large values of $\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}$ useless for the training to discourage from drastic changes.

Step 4 – Clipped Surrogate Objective

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)} \min\left[\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y), \omega(x,y)\hat{A}(x,y)\right]$$

$$\omega(x,y) = \text{clip}\left(\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}, 1-\varepsilon, 1+\varepsilon\right)$$

Proximal Policy Optimization (PPO)

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)} \min\left[\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y), \omega(x,y)\hat{A}(x,y)\right]$$

$$\omega(x,y) = \text{clip}\left(\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}, 1-\varepsilon, 1+\varepsilon\right)$$

And that's PPO!

Per-token PPO

$$\mathcal{L} = \mathbb{E}_{x\sim\mathcal{D}}\,\mathbb{E}_{\mathbf{y_{\leqslant t}}\sim\pi_{\mathbf{old}}(\mathbf{y}|\mathbf{x})}\min\left[\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\text{old}}(y_t|x,y_{<t})}\widehat{A}_t(x,y_{<t},y_t),\text{clip}\left(\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{\text{old}}(y_t|x,y_{<t})},1-\varepsilon,1+\varepsilon\right)\widehat{A}_t(x,y_{<t},y_t)\right] \approx$$

$$\approx \frac{1}{|B|}\sum_{x\in B, y\sim\pi_{old}(y|x)}\frac{1}{\text{len}(y)}\sum_{t=1}^{\text{len}(y)}\min\left[\ldots\right]$$

A note about clipping

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)} \min\left[\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y), \omega(x,y)\hat{A}(x,y)\right]$$

$$\omega(x,y) = \text{clip}\left(\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}, 1-\varepsilon, 1+\varepsilon\right)$$

This loss makes it only relevant for $\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}$ to be in $[1-\varepsilon, 1+\varepsilon]$. So:

$$(1-\varepsilon)\pi_\theta^{old}(y|x) \leq \pi_\theta(y|x) \leq (1+\varepsilon)\pi_\theta^{old}(y|x)$$

So, frequent events are boosted, while rare don't get enough motivation to manifest.

A note about clipping

$$\mathcal{L}_{RL} = \mathbb{E}_{x\sim\mathcal{D}}\mathbb{E}_{y\sim\pi_\theta^{old}(y|x)} \min\left[\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}\hat{A}(x,y), \omega(x,y)\hat{A}(x,y)\right]$$

$$\omega(x,y) = \text{clip}\left(\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}, 1-\varepsilon_{low}, 1+\varepsilon_{high}\right)$$

This loss makes it only relevant for $\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}$ to be in $[1-\varepsilon, 1+\varepsilon]$. So:

$$(1-\varepsilon)\pi_\theta^{old}(y|x) \leq \pi_\theta(y|x) \leq (1+\varepsilon)\pi_\theta^{old}(y|x)$$

So, frequent events are boosted, while rare don't get enough motivation to manifest.

DAPO https://arxiv.org/abs/2503.14476

Group Relative Policy Optimization (GRPO)

$$\mathcal{L}_{RL} = \frac{1}{G}\sum_{i=1}^{G} \sum_{y_i \sim \pi_\theta^{old}(y_i|x_i)} \min\left[\frac{\pi_\theta(y_i|x_i)}{\pi_\theta^{old}(y_i|x_i)}\hat{A}_i, \omega(x_i, y_i)\hat{A}_i\right]$$

$$\omega(x, y) = \text{clip}\left(\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}, 1-\varepsilon, 1+\varepsilon\right)$$

Advantage in PPO: $\hat{A}(x, y) = R(x, y) - \hat{V}(x)$ is called

Group Relative Policy Optimization (GRPO)

$$\mathcal{L}_{RL} = \frac{1}{G}\sum_{i=1}^{G} \sum_{y_i \sim \pi_\theta^{old}(y_i|x_i)} \min\left[\frac{\pi_\theta(y_i|x_i)}{\pi_\theta^{old}(y_i|x_i)}\hat{A}_i, \omega(x_i, y_i)\hat{A}_i\right]$$

$$\omega(x, y) = \text{clip}\left(\frac{\pi_\theta(y|x)}{\pi_\theta^{old}(y|x)}, 1-\varepsilon, 1+\varepsilon\right)$$

Advantage in GRPO: $\hat{A}_i = \frac{R_i - mean(R_1,\ldots,R_G)}{std(R_1,\ldots,R_G)+\varepsilon}$

Cases of RL in mathematics

Andrews-Curtis conjecture

Groups, generators, and relations

A group G is given by generators and relations:

$$G = \langle x_1, \dots, x_m | \, r_1, \dots, r_n \rangle$$

- Every element of the group is a word in x_i and x_i^{-1}
- r_j are words said to be the identity

Example. $G = \langle x, y | xy, x^2y \rangle$ means that $xy = x^2y = e$

Groups, generators, and relations

Question. Can we decide from the relations that $G = \langle x_1, \dots, x_m | \, r_1, \dots, r_n \rangle$ whether G is trivial ($G = \{e\}$) or not?

Groups, generators, and relations

Question. Can we decide from the relations that $G = \langle x_1, \dots, x_m | \, r_1, \dots, r_n \rangle$ whether G is trivial ($G = \{e\}$) or not?

Answer. No (Novikov)

Groups, generators, and relations

Question. Can we decide from the relations that $G = \langle x_1, \dots, x_m | \, r_1, \dots, r_n \rangle$ whether G is trivial ($G = \{e\}$) or not?

Answer. No (Novikov)

Andrews-Curtis conjecture. If $m = n$ (a balanced representation) and $G = \langle e \rangle$, then r_i can be transformed into x_i through a finite sequence of **Andrews-Curtis moves**:

(AC1) Substitute r_i by $r_i r_j$ ($i \neq j$)

(AC2) Replace r_i by r_i^{-1}

(AC3) Replace some r_i by $g r_i g^{-1}$ where $g = r_j$ or r_j^{-1}

Akbulut-Kirby groups

Akbulut-Kirby groups are considered potential counterexamples.

$$AK(n) = \langle x, y \,|\, x^n = y^{n+1}, xyx = yxy \rangle$$

Length: $n + (n+1) + 3 + 3 = 2n + 7$. Can be reduced through AC moves to $n + 11$:

$$AK(n) \cong \langle x, y, \,|\, x^{-1}yx = xyx^{-1}y, \qquad xy^{n-1}x = yxy \rangle$$

Can we reduce the length even further?

Miller-Schupp groups

$$MS(n, w) = \langle x, y \,|x^{-1}y^n x = y^{n+1}, x = w\rangle,$$

where w is a work with exponent 0 on x.

Theorem (Myasnikov, Myasnikov, Shpilrein)

$$AK(n) \sim_{AC} MS(n, y^{-1}x^{-1}yxy)$$

Traversing the Andrews-Curtis graph

- Vertices are sets of relations
- Arrows are AC moves

They bound it by presentation length

Breadth-first search with visit cap 10^6

Greedy search: similar, but states are ordered by (k, l):

- k = length of the current presentation,
- l = path length from the initial presentation.

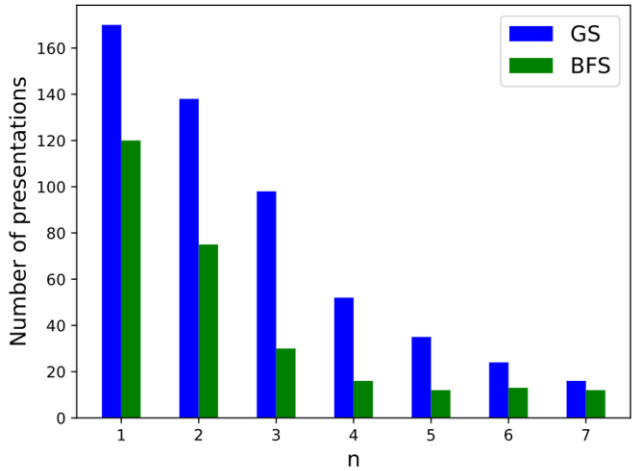
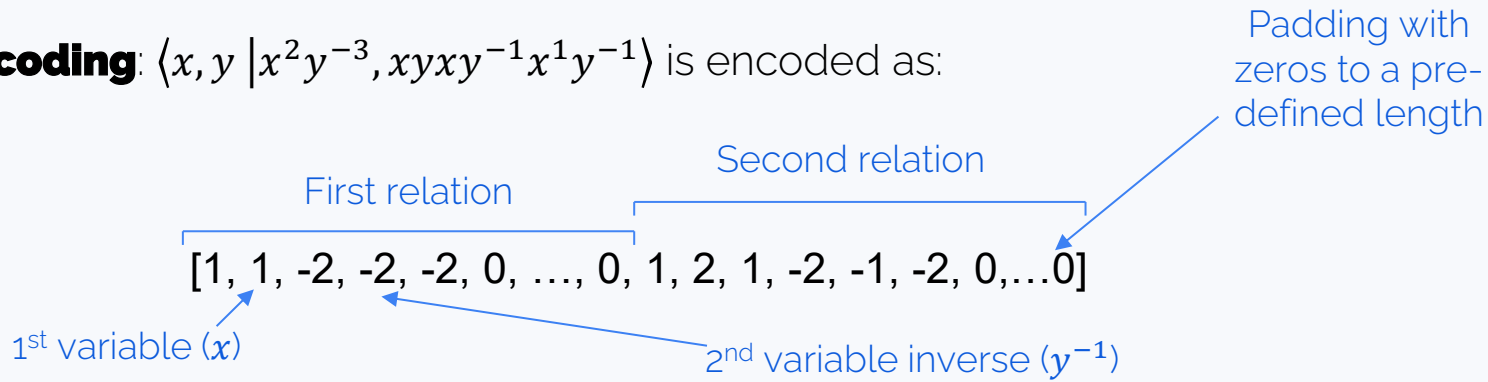


FIGURE 1. Comparison of greedy and breadth-first search algorithms as a function of n. The number of presentations of the Miller–Schupp series, $\mathrm{MS}(n, w)$, solved by an algorithm is given on the vertical axis.

What makes math problems hard for reinforcement learning: a case study https://arxiv.org/abs/2408.15332

A RL approach: PPO

State encoding: $\langle x, y \,|x^2y^{-3}, xyxy^{-1}x^1y^{-1}\rangle$ is encoded as:



Architectures of both policy and V (actor and critic):

```
Linear(input_dim → 512)
Tanh
Linear(512 → 512)
Tanh
Linear(512 → output_dim)
```

Policy predicts one of 12 actions

V predicts a scalar value (regression)

What makes math problems hard for reinforcement learning: a case study https://arxiv.org/abs/2408.15332

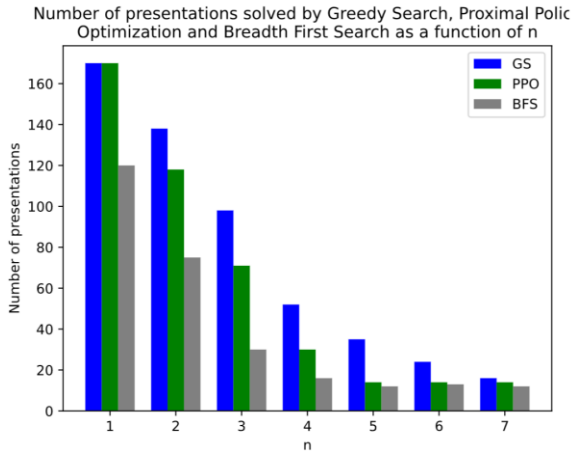
A RL approach

Train on: $MS(n, w)$, $n \leq 7$, $length(w) \leq 7$

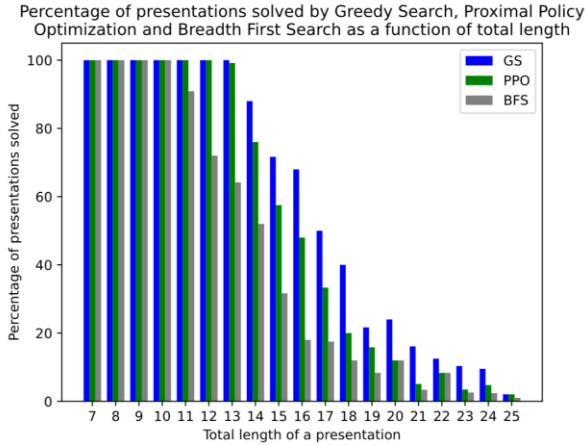
- First, all $\mathbf{1190}$ presentations are selected once, in ascending order by n and $length(w)$.
- After that, they sample from the solved set with probability $\mathbf{1/4}$ and from the unsolved set with probability $\mathbf{3/4}$.

Horizon length: the number T of AC moves the agent is allowed to take

The best-performing variable-horizon PPO agent solved two Miller–Schupp presentations that greedy search had failed to solve



(A) Distribution versus n



(B) Distribution versus length

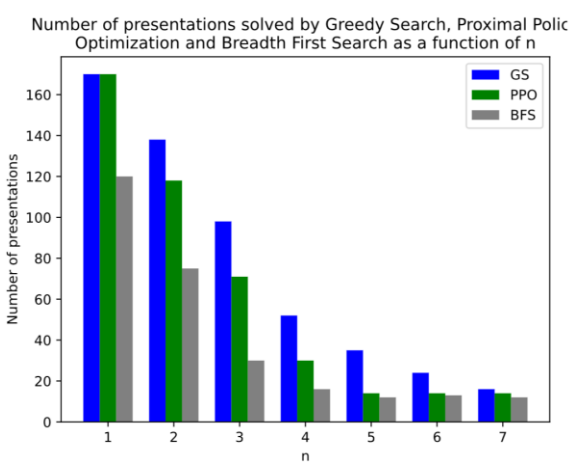
Why is it hard?

- Length is a good proxy, but it might be necessary to increase it along the way (Example from the paper: start with 17 and 19, maximum of 29 and 30.)
- Path might be long (hundreds easily), so the reward is sparse

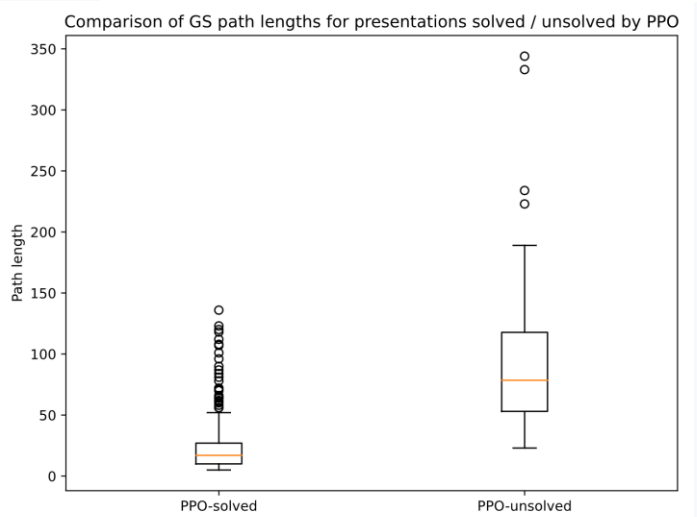
Lisitsa used automatic provers for the same task; he reported a trivialization of

$$MS_9(w^*) = \langle x, y \,|x^{-1}y^9xy^{-10}, x^{-1}y^{-1}xyx^{-1}\rangle$$

The sequence had 8634 elementary moves, reportedly requiring about 74 days and over 56 GB of memory



(A) Distribution versus n



Proposed counter-measure: super moves

Some repeated patterns can be reassigned as new atomic actions

Technically:

- Your policy has several action slots that are masked in the beginning
- As new super moves are discovered, these slots are unlocked

What makes math problems hard for reinforcement learning: a case study https://arxiv.org/abs/2408.15332

Takeaways

Some theorems were proved:

Theorem B. *The following infinite subfamilies of Miller–Schupp presentations are AC-trivial:*

$$\begin{aligned} &(i)\ \ MS(1, w) \text{ for all } w, \\ &(ii)\ \ MS(n, w_\star) \text{ for all } n > 0 \text{ and } w_\star = y^{-1}xyx^{-1}, \\ &(iii)\ \ MS(2, w_k) \text{ for } w_k = y^{-k}x^{-1}yxy \text{ where } k \in \mathbb{Z}. \end{aligned}$$

Furthermore, for each fixed $n > 0$, all members of the 1-parameter family, $MS(n, w_k)$, parameterized by $k \in \mathbb{Z}$, are AC-equivalent to each other and to $AK(n)$.

What makes math problems hard for reinforcement learning: a case study https://arxiv.org/abs/2408.15332

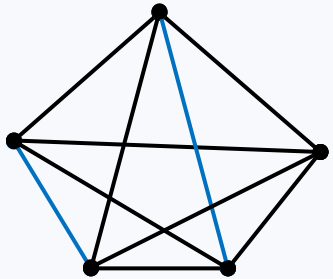
Extremal constructions in combinatorics via RL

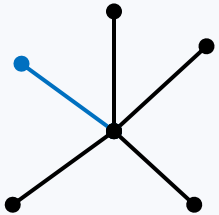
A graph problem

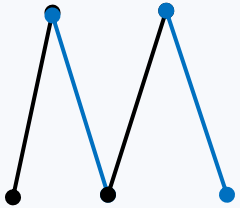
Conjecture 1. Let G be a connected graph on $n \geq 3$ vertices, with largest eigenvalue λ_1 and matching number μ. Then

$$\lambda_1 + \mu \geq \sqrt{n-1} + 1$$

- λ_1 is the largest eigenvalue of the adjacency matrix
- μ is the maximal number of non-adjacent edges







Graph	λ_1	μ
Path P_n	$2\cos\left(\frac{\pi}{n+1}\right)$	$\left\lfloor\frac{n}{2}\right\rfloor$
Star $K_{1,n-1}$	$\sqrt{n-1}$	1
Complete graph K_n	$n-1$	$\left\lfloor\frac{n}{2}\right\rfloor$
Bipartite graph $K_{a,b}$	$\sqrt{ab}$	$\min(a,b)$

A graph problem

Conjecture 1. Let G be a connected graph on $n \geq 3$ vertices, with largest eigenvalue λ_1 and matching number μ. Then

$$\lambda_1 + \mu \geq \sqrt{n-1} + 1$$

- λ_1 is the largest eigenvalue of the adjacency matrix
- μ is the maximal number of non-adjacent edges

Kinda we're searching for a graph with min matching (rather dense) and slow spread.

Graph	$\boldsymbol{\lambda_1}$	$\boldsymbol{\mu}$
Path P_n	$2\cos\left(\frac{\pi}{n+1}\right)$	$\left\lfloor\frac{n}{2}\right\rfloor$
Star $K_{1,n-1}$	$\sqrt{n-1}$	1
Complete graph K_n	$n-1$	$\left\lfloor\frac{n}{2}\right\rfloor$
Bipartite graph $K_{a,b}$	$\sqrt{ab}$	$\min(a,b)$

A graph problem

Conjecture 1. Let G be a connected graph on $n \geq 3$ vertices, with largest eigenvalue λ_1 and matching number μ. Then

$$\lambda_1 + \mu \geq \sqrt{n-1} + 1$$

- λ_1 is the largest eigenvalue of the adjacency matrix
- μ is the maximal number of non-adjacent edges

We want to disprove it: find a graph such that

$$\lambda_1 + \mu < \sqrt{n-1} + 1$$

It's a combinatorial optimization problem

Graph	$\boldsymbol{\lambda_1}$	$\boldsymbol{\mu}$
Path P_n	$2\cos\left(\frac{\pi}{n+1}\right)$	$\left\lfloor \frac{n}{2} \right\rfloor$
Star $K_{1,n-1}$	$\sqrt{n-1}$	1
Complete graph K_n	$n-1$	$\left\lfloor \frac{n}{2} \right\rfloor$
Bipartite graph $K_{a,b}$	$\sqrt{ab}$	$\min(a,b)$

Exploration and exploitation with VNS

Optimization problem. Find a graph G such that

$$\lambda_1 + \mu < \sqrt{n-1} + 1$$

Previous result: a graph on 600 vertices ($n = 600, \mu = 8$, T a path on 8 vertices) using Variable Neighborhood Search metaheuristic.

"Neighborhood" might be

- Add or delete one edge.
- Add a pendant vertex
- Replace a subgraph by another subgraph of the same size.
- Whatever, honestly

Define "neighborhoods" $N_1, N_2, \ldots, N_k$
Set $i = 1$

Then repeat:

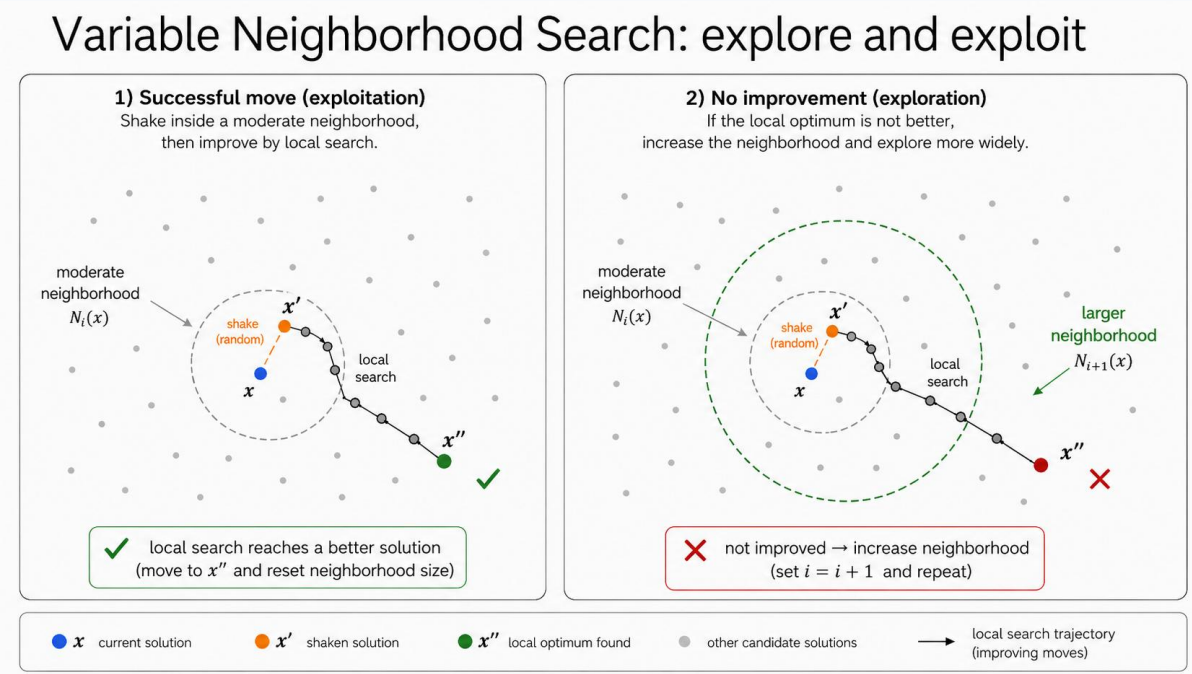
- Pick a random graph x' from $N_i(x)$ ("shaking")
- Run local search starting from x', arriving at a local optimum x''.
- If x'' is better than x, replace x by x'' and go back to $i = 1$.
- Otherwise increase $i = i + 1$, trying a broader/different neighbourhood.

Resolution of AutoGraphiX conjectures relating the index and matching number of graphs.
https://arxiv.org/abs/2408.15332

Exploration and exploitation with VNS

Optimization problem. Find a graph G such that

$$\lambda_1 + \mu < \sqrt{n-1} + 1$$



Resolution of AutoGraphiX conjectures relating the index and matching number of graphs.
https://arxiv.org/abs/2408.15332

A graph problem

Optimization problem. Find a graph G such that

$$\lambda_1 + \mu < \sqrt{n-1} + 1$$

Previous result: a graph on 600 vertices ($n = 600, \mu = 8$, T a path on 8 vertices) using Variable Neighborhood Search metaheuristic.

Let n and μ be chosen freely, such that n is sufficiently larger than μ. Let $S_1, S_2, \ldots, S_\mu$ be the stars, each with either $\lfloor \frac{n-1}{\mu} \rfloor + 1$ or $\lceil \frac{n-1}{\mu} \rceil + 1$ vertices, so that their total number of vertices is $n + \mu - 1$. Further, let T be an arbitrary tree on vertex set $\{1, 2, \ldots, \mu\}$, with its maximum degree being at most $\lfloor \frac{n-1}{\mu} \rfloor$. The new tree T^* is formed from T first by replacing vertex i of T with star S_i, for each $1 \leq i \leq \mu$, and then by identifying a distinct pair of leaves, one from S_i and one from S_j, for each edge ij in T. An example of tree T^* obtained for $n = 17, \mu = 4$ and T being a star on four vertices, is shown in Fig. 1.

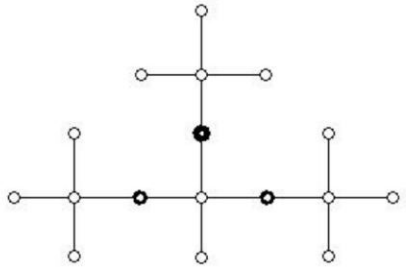


Figure 1: An example of tree T^*. Darker vertices represent pairs of leaves that were identified in forming T^*.

Resolution of AutoGraphiX conjectures relating the index and matching
https://arxiv.org/abs/2408.15332

Cross-entropy method

VNS vs Cross-Entropy at high level

VNS:

- Start from a small neighborhood, try searching there;
- If you fail – expand/change the neighborhood, search there;
- If you find a better example, shrink the neighborhood again, search closely there

But which neighborhoods to choose – it's totally your responsibility.

Cross-Entropy method:

- We'll train an agent to construct graphs maximizing $\lambda_1 + \mu$
- We'll train it on better and better subsets of graphs on n vertices
- Defining these better and better subsets will be a part of the training (no hand-written heuristics, yay!)

Cross-entropy method for this problem

The agent: fills in the graph, edge by edge, producing:

1, if the edge w_2 is present
0, otherwise

$$graph = (w_1, w_2, \dots, w_n) \in \{0,1\}^{\binom{n}{2}}$$

Its input, at i-th step:

1-st edge is present
2-nd edge is absent

Not considered yet

1 at the i-th position

$$x_i = (\underbrace{1, 0, 1, 0, 0, 0, 0, \dots}_{\text{Edges}}, \underbrace{0, 0, 0, 0, 1, 0, 0, \dots}_{\text{Current position encoding}}) \in \{0,1\}^{2\cdot\binom{n}{2}}$$

The agent is a NN with ReLU activations: $342 \to 128 \to 64 \to 4 \to 1$.
Predicts 0 or 1 on each step.

How do we train it?!

Train on elite set

Idea: Let's proceed in "epochs" (sampled sets of graphs).
One epoch = we let our policy freely generate N (say, 1000) graphs $G_1, \dots, G_N$ through sampling:

$$x_{i+1} \sim p_\theta(x_i)$$

After each "epoch", compute rewards $R_i = R(G_i) = \sqrt{n-1} + 1 - \lambda_1(G_i) - \mu(G_i)$.

Choose an elite set G_i with top 5% rewards and only train on them

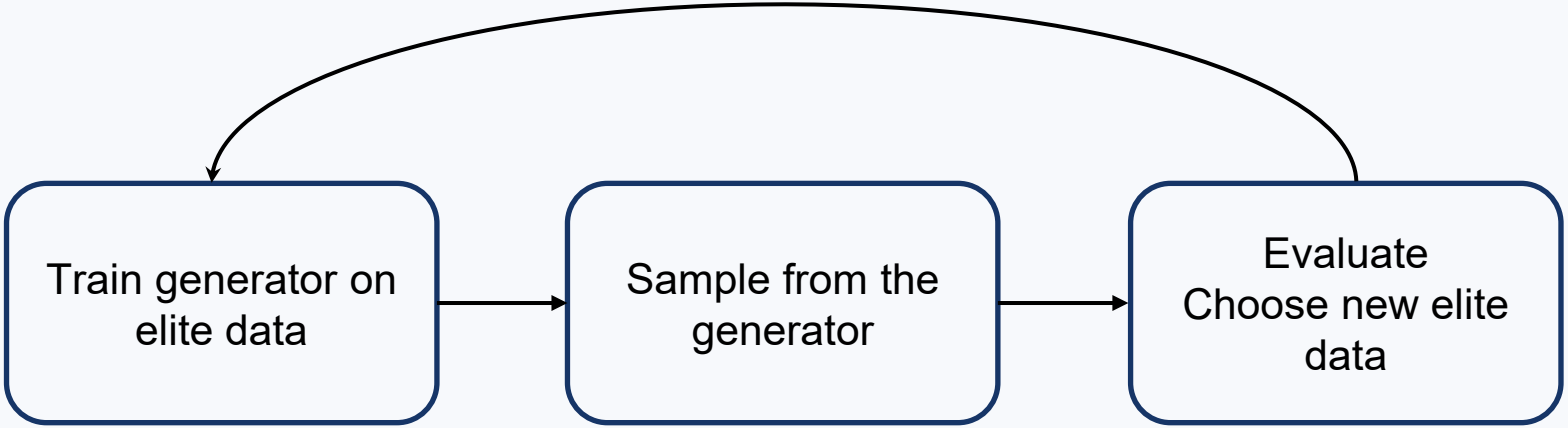
$$\mathcal{L} = -\sum_{i \in top\ 5\%} w_i^{true} \log p_\theta(x_i) + \left(1 - w_i^{true}\right)(1 - \log p_\theta(x_i))$$

General Cross-Entropy method

We have a generator π_θ of the object of interest + some score function.

Iteratively:

- Generate N objects with π_θ.
- Score each object.
- Select the **elite set**: $S_t \subset \{x_1, \dots, x_N\}$ consisting of the top $\rho \cdot N$ objects (e. g. $\rho = 0.1$).
- Update policy parameters using only the elite objects.

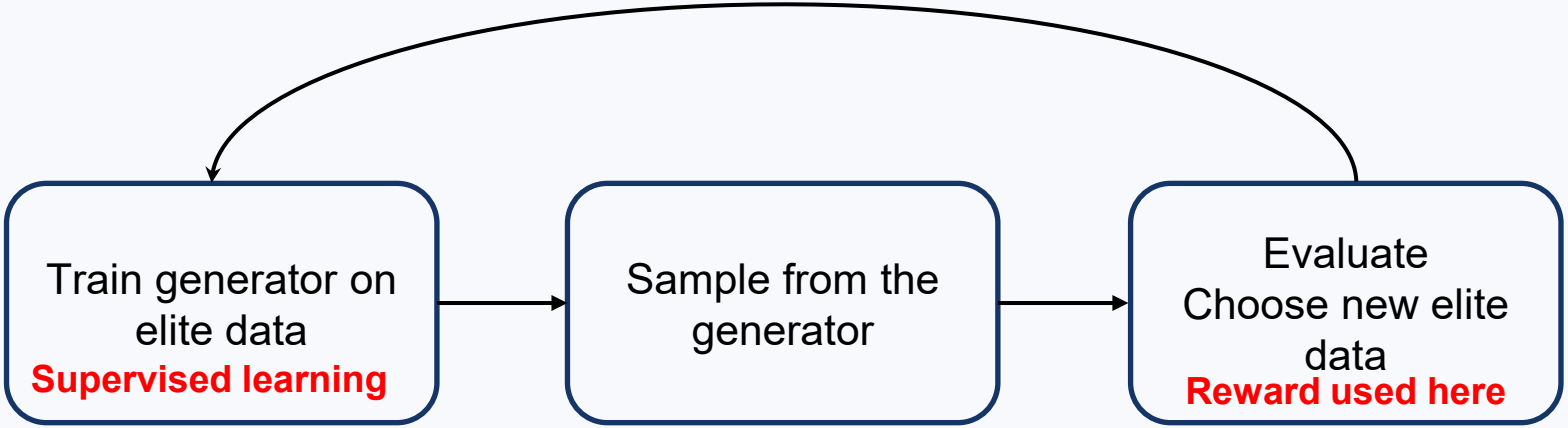


Cross-Entropy vs RL

We have a generator π_θ of the object of interest + some score function.

Iteratively:

- Generate N objects with π_θ.
- Score each object.
- Select the **elite set**: $S_t \subset \{x_1, \ldots, x_N\}$ consisting of the top $\rho \cdot N$ objects (e. g. $\rho = 0.1$).
- Update policy parameters using only the elite objects.



Results for λ_1 and μ

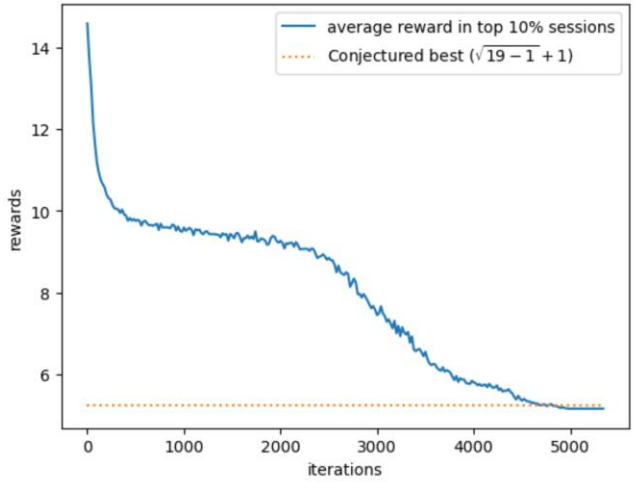


Figure 2: The average value of $\lambda_1 + \mu$ of the best 10% of the sessions in each iteration decreases over time. After 5000 iterations it goes slightly below the conjectured threshold of $\sqrt{19-1}+1$, meaning we have found a counterexample to Conjecture 2.1.

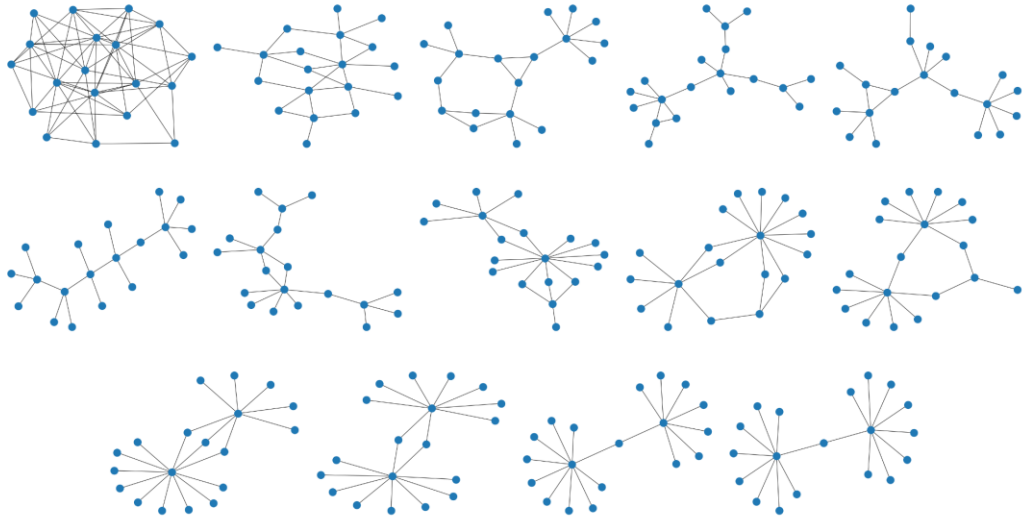


Figure 3: The evolution of the best construction over time. The network quickly realizes that sparse graphs are best, and eventually the "balanced double star" structure emerges.

Constructions in combinatorics via neural networks
https://arxiv.org/abs/2104.14516

Reproducing the paper

What we tried:

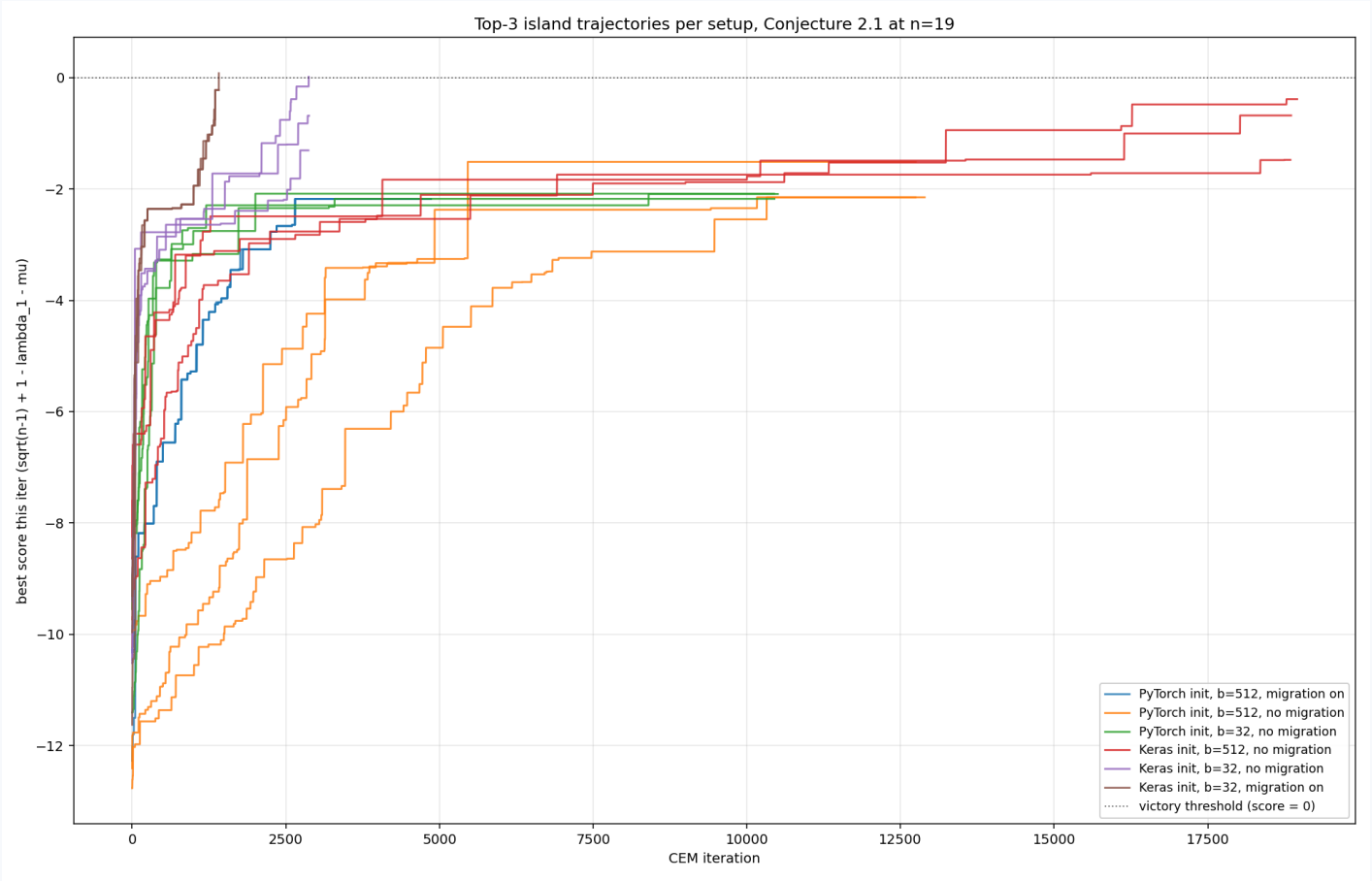
- Batch size 32 vs 512
- We work with several "islands" – they may evolve independently, but we can also have migration between them to mix the best examples (or best + random examples, actually, but I never implemented it)
- Initialization: Keras-style Xavier

$$W \sim Uniform\left(-\sqrt{\frac{6}{d_{in}+d_{out}}}, \sqrt{\frac{6}{d_{in}+d_{out}}}\right), \qquad b = 0$$

vs Pytorch-style Kaiming:

$$W \sim Uniform\left(-\frac{1}{\sqrt{d_{in}}}, \frac{1}{\sqrt{d_{in}}}\right), \qquad b = Uniform\left(-\frac{1}{\sqrt{d_{in}}}, \frac{1}{\sqrt{d_{in}}}\right)$$

Reproducing the paper



Let's check github

Another example

Proximity of a graph G:

$$\pi(G) = \frac{1}{n-1} \min_{v \in V(G)} \sum_{w \in V(G)} d(v,w)$$

$\partial_1 \geq \partial_2 \geq \cdots$ - eigenvalues of the distance matrix $D(G) = \big(d(v,w)\big)_{v,w}$

We know, for a connected graph with diameter D: $\pi \geq 1$, $\partial_{\lfloor D/2 \rfloor} \geq -1$
Hence:

$$\pi + \partial_{\lfloor D/2 \rfloor} \geq 0$$

Conjecture (Aouchiche-Hansen):

$$\pi + \partial_{\lfloor 2D/3 \rfloor} \geq 0$$

Another example

Conjecture (Aouchiche-Hansen):

$$\pi + \partial_{\lfloor 2D/3 \rfloor} \geq 0$$

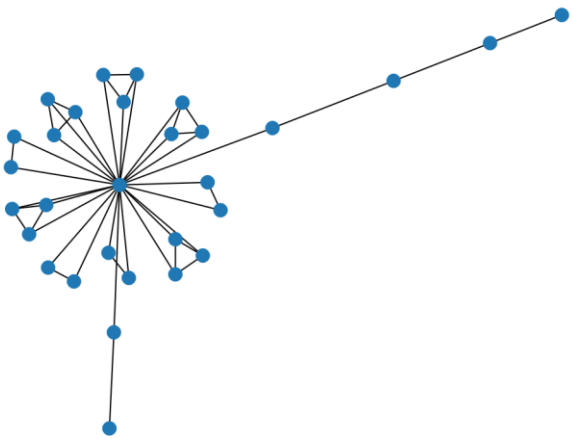


Figure 5: The graph on 30 vertices with smallest value of $\pi + \partial_{\lfloor \frac{2D}{3} \rfloor}$ found by the network. It has $\pi + \partial_{\lfloor \frac{2D}{3} \rfloor} \approx 0.4$ so it is not quite a counterexample to Conjecture 2.3, but it tells us very clearly what counterexamples could look like.

Constructions in combinatorics via neural networks https://arxiv.org/abs/2104.14516

Another example

Conjecture (Aouchiche-Hansen):

$$\pi + \partial_{\lfloor 2D/3 \rfloor} \geq 0$$

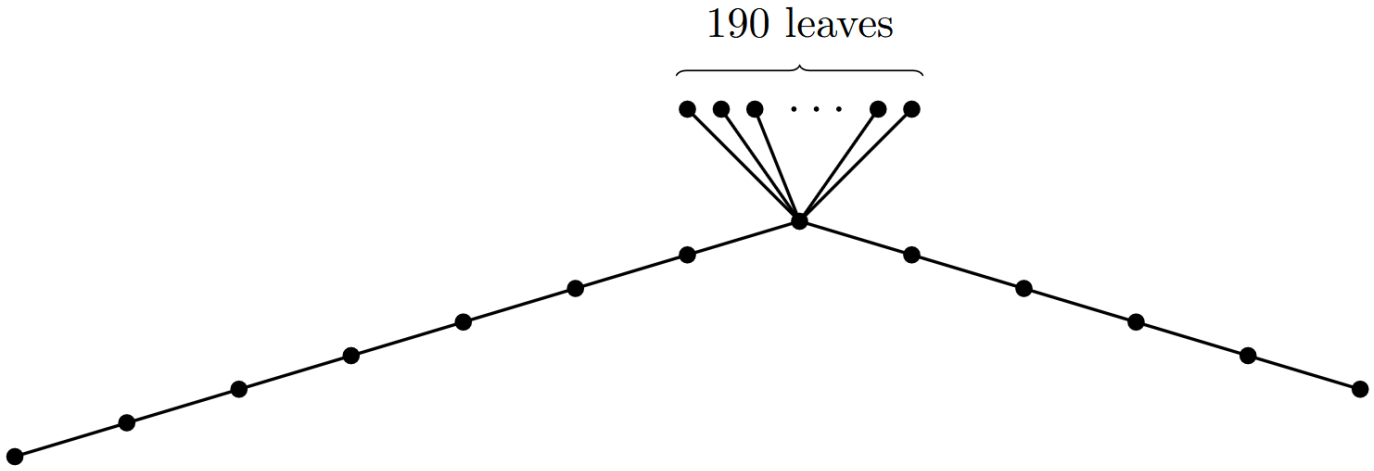


Figure 6: A counterexample to Conjecture 2.3.

Constructions in combinatorics via neural networks https://arxiv.org/abs/2104.14516

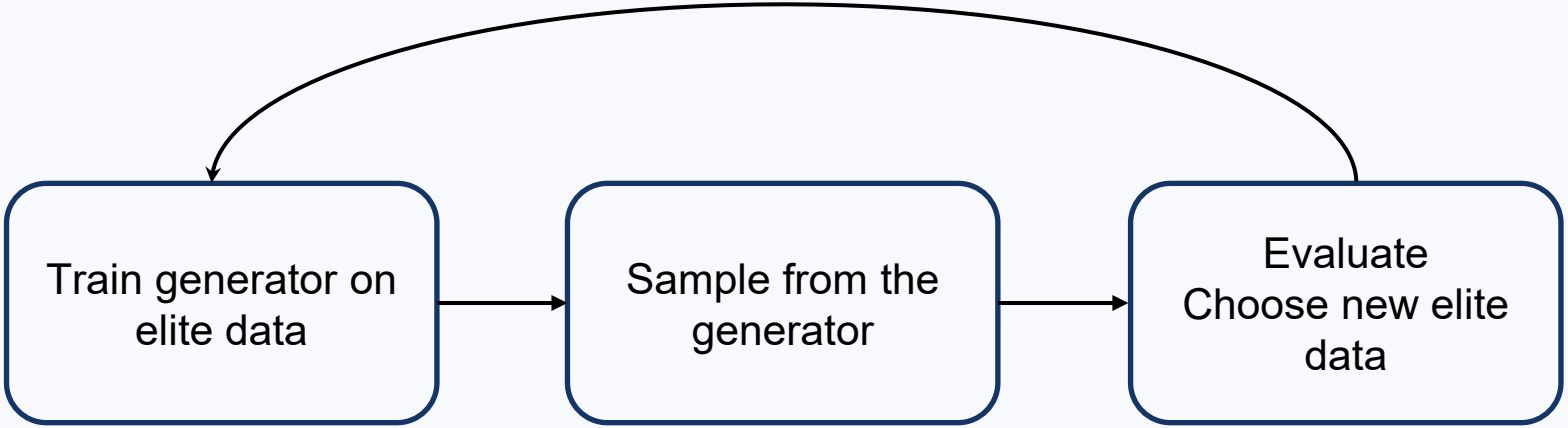
PatternBoost

General Cross-Entropy method

We have a generator π_θ of the object of interest + some score function.

Iteratively:

- Generate N objects with π_θ.
- Score each object.
- Select the **elite set**: $S_t \subset \{x_1, \dots, x_N\}$ consisting of the top $\rho \cdot N$ objects (e. g. $\rho = 0.1$).
- Update policy parameters using only the elite objects.

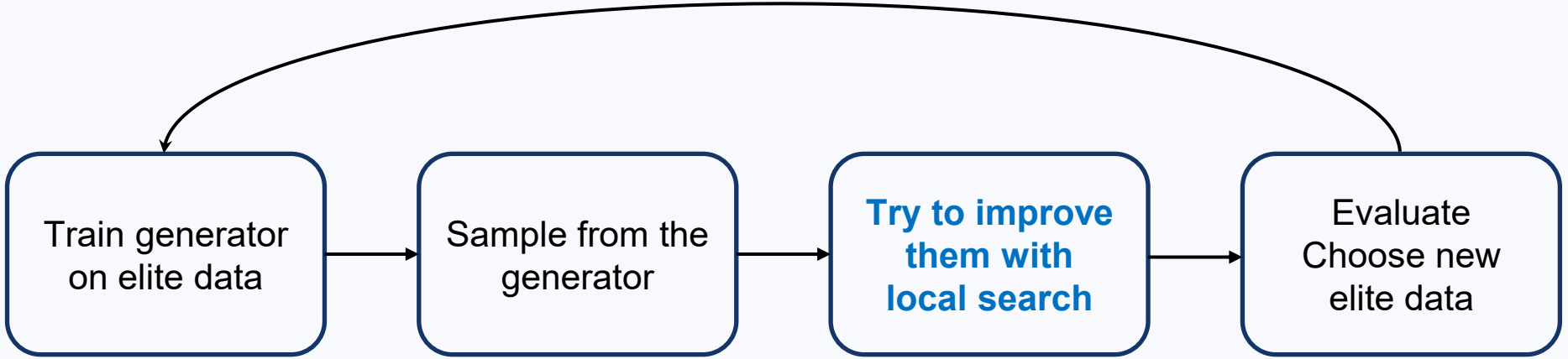


Constructions in combinatorics via neural networks https://arxiv.org/abs/2104.14516

PatternBoost

We have a generator π_θ of the object of interest + some score function.
Iteratively:

- Generate N objects with π_θ.
- **Try to improve them with local search**
- Score each of the improved objects.
- Select the **elite set**: $S_t \subset \{x_1, \ldots, x_N\}$ consisting of the top $\rho \cdot N$ objects (e. g. $\rho = 0.1$).
- Update policy parameters using only the elite objects.



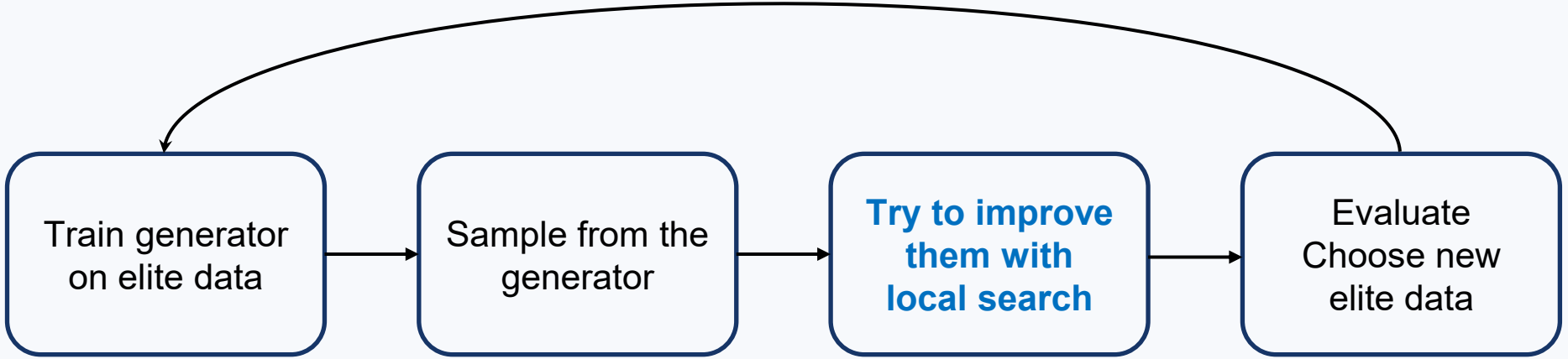
PatternBoost: Constructions in Mathematics with a Little Help from AI https://arxiv.org/abs/2411.00566v1

Example: discovering bipartite graphs

Problem: On a fixed number n of vertices, find a graph with **many edges** and **no triangles**.

$$score(G) = \#E(G) - \lambda \cdot \#\, triangles(G)$$

How do we generate graphs?



PatternBoost: Constructions in Mathematics with a Little Help from AI https://arxiv.org/abs/2411.00566v1

Example: discovering bipartite graphs

Problem: On a fixed number n of vertices, find a graph with **many edges** and **no triangles**.

$$score(G) = \#E(G) - \lambda \cdot \#\, triangles(G)$$

Let's generate graphs as **adjacency matrices**.

$$\text{"010,01,0"} \rightarrow \begin{pmatrix} 0 & 1 & 0 \\ & 0 & 1 \\ & & 0 \end{pmatrix} \rightarrow \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}$$

They use **BPE tokenization** with a vocabulary size of 100.

The generative model is a **decoder-only transformer**:

- 2 layers
- 4 attention heads
- embedding dimension 16

PatternBoost: Constructions in Mathematics with a Little Help from AI https://arxiv.org/abs/2411.00566v1

Example: discovering bipartite graphs

Problem: On a fixed number n of vertices, find a graph with **many edges** and **no triangles**.

$$score(G) = \#E(G) - \lambda \cdot \#\, triangles(G)$$

Local search: a very simple idea:

- remove one edge from each triangle in G
- then randomly add edges without creating triangles (if possible)

PatternBoost: Constructions in Mathematics with a Little Help from AI https://arxiv.org/abs/2411.00566v1

Let's look at the code!

A genuine discovery

Problem: What is the maximum number of edges one can delete from the d-dimensional hypercube, without increasing its diameter?

Previous conjecture: $\frac{1}{2}2^d \cdot \left(2^d - 1\right) - \left[2^d + \binom{d}{\lfloor \frac{d}{2} \rfloor} - 2\right]$

New result: for $d = 6$ a graph $81 < 82$ remaining edges

PatternBoost: Constructions in Mathematics with a Little Help from AI https://arxiv.org/abs/2411.00566v1

No isosceles triangles

Problem: On a $n \times n$ grid what is a maximal number of points we could place so that there'd be no isosceles triangles among them?

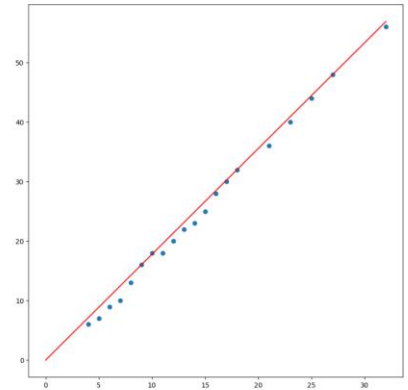


Figure 13: Plotting the computed values, f seems to be linear

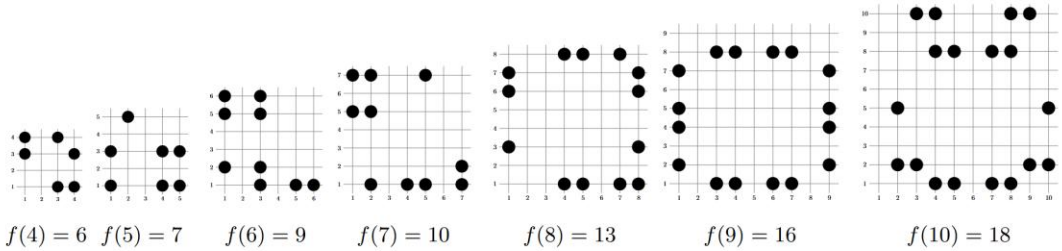


Figure 11: The best constructions for $n = 4$ to 10

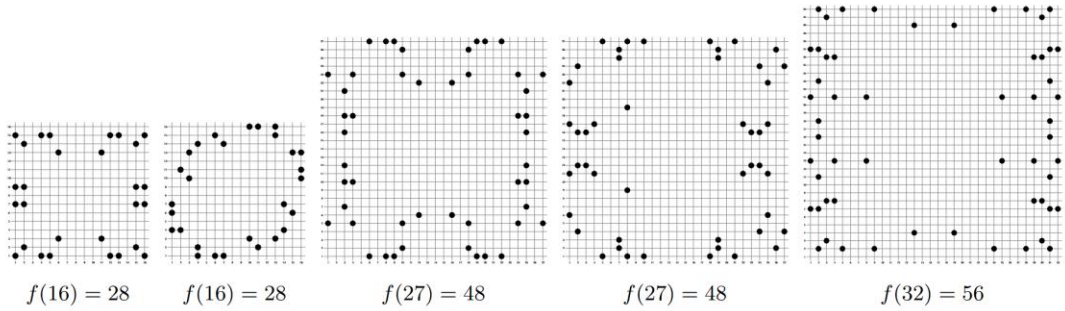


Figure 12: The best constructions for $n = 16$, 27, and 32. For some n, there are many optimal solutions that look very different from each other.

No isosceles triangles

Problem: On a $n \times n$ grid what is a maximal number of points we could place so that there'd be no isosceles triangles among them?

n	Standard search	PatternBoost
64	108	110
100	154	160

PatternBoost: Constructions in Mathematics with a Little Help from AI https://arxiv.org/abs/2411.00566v1

Hadamard matrices

Hadamard matrix: An $n \times n$ matrix with elements $\{\pm 1\}$ and mutually orthogonal rows

$S = -\log\det\left(\frac{1}{n^{\frac{1}{2}}} M\right) \geq 0$ for $\{\pm 1\}$ matrices

It's $= 0$ for Hadamard matrices

Search among Goethas-Seidel matrices

$$M = \begin{bmatrix} A & BF & CF & DF \\ -BF & A & -FD & FC \\ -CF & FD & A & -FB \\ -DF & -FC & FB & A \end{bmatrix}$$

where A, B, C, D are circulant matrices

$$F = \begin{pmatrix} 0 & \cdots & 1 \\ \vdots & \iddots & \vdots \\ 1 & \cdots & 0 \end{pmatrix}$$

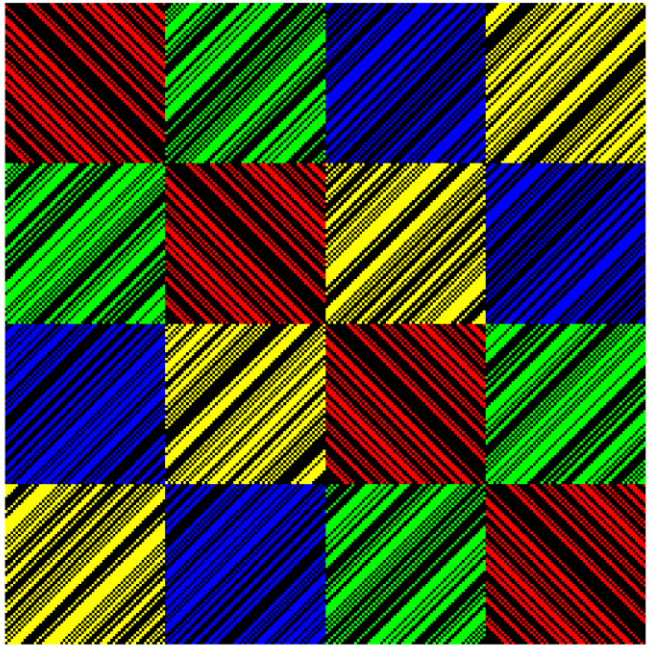


FIGURE 1. A GS type Hadamard matrix of order 244. Coloured entries are $+1$, black entries are -1.

Generating Hadamard matrices with transformers https://arxiv.org/pdf/2604.11101v2

Fourier sorcery

Hadamard matrix: An $n \times n$ matrix with elements $\{\pm 1\}$ and mutually orthogonal rows

For $\{\pm 1\}$ GS matrices

$$0 \leq S = -\log\det\left(\frac{1}{n^{\frac{1}{2}}}M\right) = -\sum_{j=0}^{n'-1}\log\left(\sum_{i=1}^{4}\left|\hat{a}_{i,j}\right|^2\right)$$

where $a_{i,j}$ are elements of the 1st row of A, B, C, D $(i = 1, .., 4)$

$$\hat{a}_{i,j} = \frac{1}{\sqrt{n}}\sum_{k=0}^{n'-1} a_{i,k}\omega^{j,k}, \qquad \omega = \exp\left(\frac{2\pi i}{n}\right)$$

Better objective:

$$\sum_{i=1}^{4}\left|\hat{a}_{i,j}\right|^2 - 1 \to 0, \qquad j = 0 \ldots n' - 1$$

$$M = \begin{bmatrix} A & BF & CF & DF \\ -BF & A & -FD & FC \\ -CF & FD & A & -FB \\ -DF & -FC & FB & A \end{bmatrix}$$

Generating Hadamard matrices with transformers https://arxiv.org/pdf/2604.11101v2

Local search

1. Flips: $a_{r,q} \to -a_{r,q}$

2. Perturb one of a_i $(i = 1..4)$ in the Fourier space, then map it back:

$$(a_1, a_2, a_3, a_4) \to \left(a_1, a_2, a_3, sign\left(\mathcal{F}^{-1}\left(\mathcal{F}(a_4)e^{i\theta_4}\right)\right)\right)$$

If we don't improve, run several actions and accept with probability

$$\mathbb{P}(accept) = \min\left(1, \exp\left(\frac{S(x') - S(x)}{T}\right)\right)$$

for some temperature T.

$$M = \begin{bmatrix} A & BF & CF & DF \\ -BF & A & -FD & FC \\ -CF & FD & A & -FB \\ -DF & -FC & FB & A \end{bmatrix}$$

Generating Hadamard matrices with transformers https://arxiv.org/pdf/2604.11101v2

Transformer

Basic version: a decoder-only transformer that builds a_1, a_2, a_3, a_4

Advanced version: the first token of each a_i gets information about Fourier features of the previous $a_1, \ldots, a_{i-1}$ (a summary vector added alongside the embedding of the 0-th position)

Generating Hadamard matrices with transformers https://arxiv.org/pdf/2604.11101v2

Shrinking search space with fixed sums

We know about $k_i = \sum_{k=0}^{n'-1} a_{i,k}$ that

$$\sum_{i=1}^{4} k_i^2 = n$$

We can fix particular decomposition of n, shrinking the search space.

	*	(5,7,7,7)	(1,5,5,11)
`False`	1 270	0 1 009	1 1 087
`True`	14 305	17 1 196	26 1 437

TABLE 1. Influence of `transformer_uses_score` (rows) and `segment_sums` (columns) on the generation of $n = 172$ Hadamard matrices: top/bottom number is pre/post improvement total number of Hadamard matrices after 30 generations.

Generating Hadamard matrices with transformers https://arxiv.org/pdf/2604.11101v2

Results

- Mass production of Hadamard matrices. For $n = 140$, they obtained $729{,}732$ Hadamard matrices, of which $729{,}715$ were inequivalent
- PatternBoost works where local search is unable to find anything

Generating Hadamard matrices with transformers https://arxiv.org/pdf/2604.11101v2

Reward-Guided Fine Tuning (FlowBoost)

Heilbronn triangle problem

Problem: Place n points in the unit square so as to maximise the area Δ_n of the smallest triangle determined by any three of the points.

What's known: for infinitely many n,

$$\Delta_n \geq c\frac{\log n}{n^2}$$

For sufficiently large N:

$$\Delta_n \leq n^{8/7-1/2000}$$

- Optimal constructions known and proved for $n = 5,6,7,8,9$
- Published constructions for $n = 10,11,12$, not necessary optimal
- Folklore constructions for $13 \leq n \leq 16$, no optimality proof

- The authors of the next paper approached $n = 13$ and $n = 15$. They didn't beat the existing numbers, but they beat simpler techniques

Heilbronn triangle problem

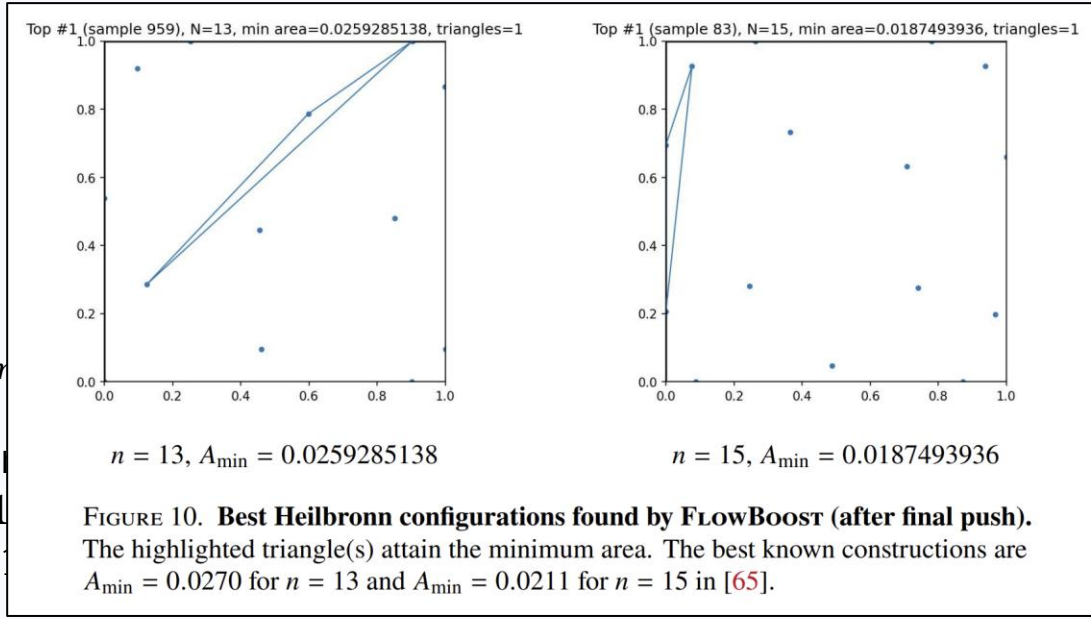
Problem: Place n points in the unit square so as to maximise the area Δ_n of the smallest triangle determined by any three of the points.

What's known: for infinitely many n,

For sufficiently large N:

Δ_n

- Optimal constructions known and pr
- Published constructions for $n = 10,1$
- Folklore constructions for $13 \leq n \leq$



$n = 13$, $A_{\min} = 0.0259285138$ $n = 15$, $A_{\min} = 0.0187493936$

FIGURE 10. **Best Heilbronn configurations found by FLOWBOOST (after final push).** The highlighted triangle(s) attain the minimum area. The best known constructions are $A_{\min} = 0.0270$ for $n = 13$ and $A_{\min} = 0.0211$ for $n = 15$ in [65].

- The authors of the next paper approached $n = 13$ and $n = 15$. They didn't beat the existing numbers, but they beat simpler techniques

Heilbronn triangle problem

Problem: Place n points in the unit square so as to maximise the area Δ_n of the smallest triangle determined by any three of the points.

A configuration is: $x = (p_1, \ldots, p_n),\ p_i = (u_i, v_i) \in [0,1]^2$

Objective to maximize is:

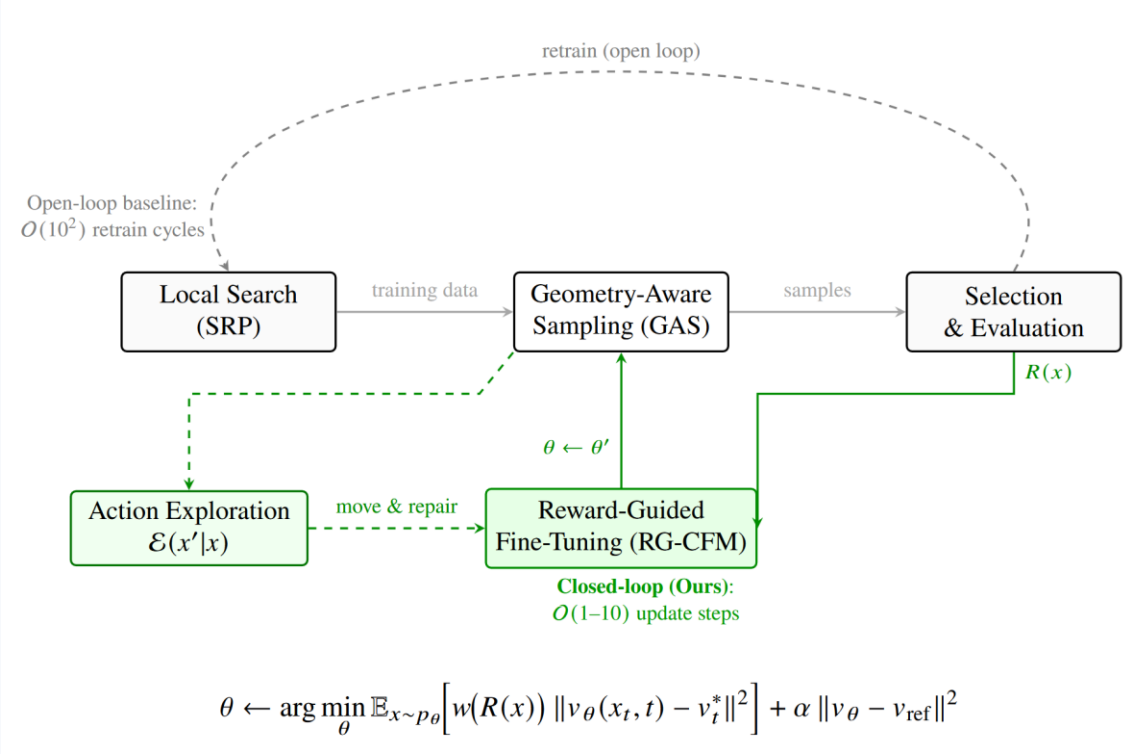
$$J(x) = \min_{1 \le i < j < k \le n} A_{ijk}$$

where

$$A_{ijk} = \frac{1}{2}\left|\det(p_j - p_i, p_k - p_i)\right|$$

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

The (ridiculously complicated) pipeline



Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Heilbronn triangle problem

Problem: Place n points in the unit square so as to maximise the area Δ_n of the smallest triangle determined by any three of the points.

A configuration is: $x = (p_1, \ldots, p_n),\ p_i = (u_i, v_i) \in [0,1]^2$

Many of them for large n

Objective to maximize is:

$$J(x) = \min_{1 \le i < j < k \le n} A_{ijk}$$

where

$$A_{ijk} = \frac{1}{2}\left|\det(p_j - p_i, p_k - p_i)\right|$$

Piecewise linear; Non-smooth

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Step 1: generate elite data by SRP
(stochastic relaxation with perturbations)

A configuration is: $x = (p_1, \dots, p_n),\ p_i = (u_i, v_i) \in [0,1]^2$

Surrogate objective to maximize is:

$$J(x) = \min_{1 \le i < j < k \le n} A_{ijk} \rightsquigarrow J_{\beta,\varepsilon}(x) = \text{softmin}\, A_{ijk} = -\frac{1}{\beta} \log \sum_{1 \le i < j < k \le n} \exp\left(-\beta A_{ijk}^{(\varepsilon)}\right)$$

where

$$A_{ijk} = \frac{1}{2}\left|D_{ijk}\right| \quad \rightsquigarrow \quad A_{ijk}^{(\varepsilon)} = \frac{1}{2}\sqrt{D_{ijk}^2 + \varepsilon^2}$$

Total loss:

$$\mathcal{L}_{SRP} = J_{\beta,\varepsilon}(x) + \lambda_W$$

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Step 1: generate elite data by SRP

(stochastic relaxation with perturbations)

A configuration is: $x = (p_1, \dots, p_n),\ p_i = (u_i, v_i) \in [0,1]^2$

Total loss:

$$\mathcal{L}_{SRP} = J_{\beta_k,\varepsilon}(x) + \lambda_W$$

$x_0 \sim Uniform([0,1]^{2n})$

Repeat:

- Perturb: $x_{k+\frac{1}{2}} = \Pi_{[0,1]^{2n}}(x + \sigma_k \xi_k)$
- Relax: $x_{k+1} = \Pi_{[0,1]^{2n}}\left(x_{k+\frac{1}{2}} - \eta_k \nabla_x \mathcal{L}_{SRP}\right)$

$\Pi_{[0,1]^{2n}}(z) = \mathrm{clip}(z, 0, 1)$

ξ_k is random

σ_k might be $\sigma_0 \rho^k$

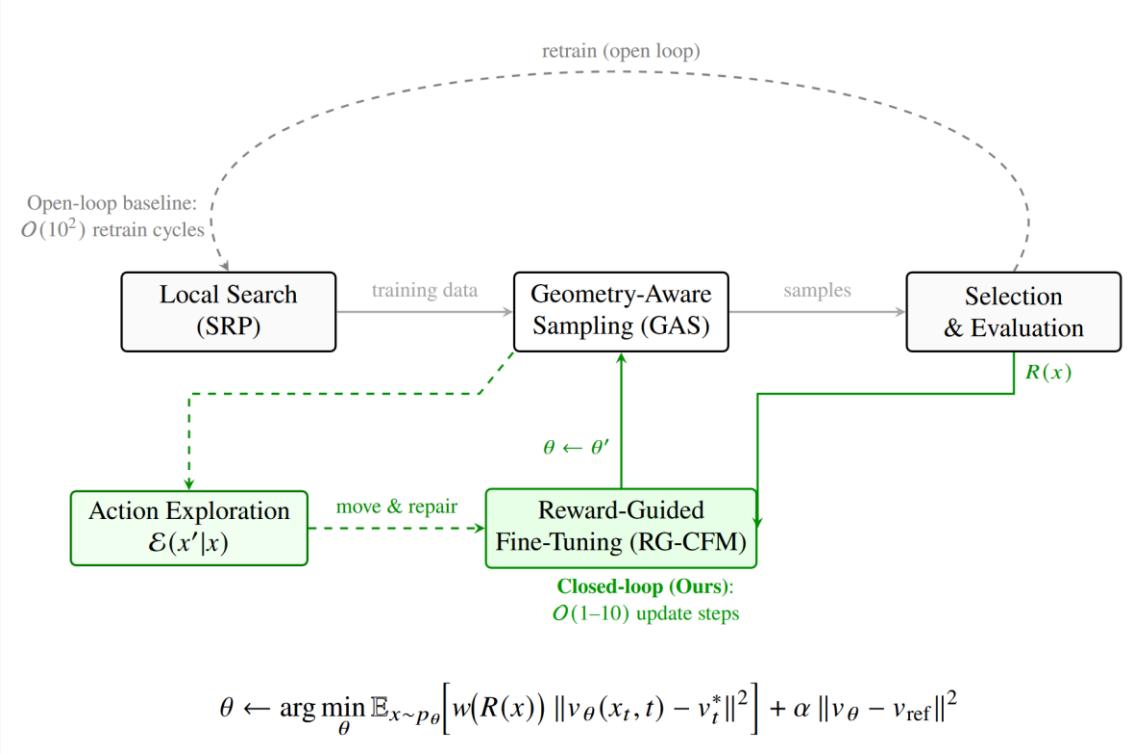
$\beta_{k+1} = \gamma_\beta \beta_k,\ \gamma > 1,$

so that softmin becomes

more and more as true min

Run from many starts; keep around 25–50% as the elite training set $\mathcal{D}_0$

The (ridiculously complicated) pipeline



Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Step 2: train a conditional flow-matching model

SRP will be back, but with a better starting distribution: instead of $x_0 \sim Uniform([0,1]^{2n})$, we'll learn $x_0 \sim p_{good}([0,1]^{2n})$

The authors use **flow matching** here. The idea is:

- Sample $x_{00} \sim Uniform([0,1]^{2n})$, interpret it as $x_0(0)$ on a curve $x_0(t)$, where we actually need $x_0(1)$, which will be good

A curve is $\dot{x} = v_\theta(x(t), t)$. Where do we get v_θ?!

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Step 2: train a conditional flow-matching model

A curve is $\dot{x} = v_\theta(x(t), t)$. Where do we get v_θ?!

Sample a random x_0 and $x_1 = x_{elite}$ from the elite set.

Let the trajectory be $x_t = (1-t)x_0 + tx_1 \Rightarrow$ the velocity is $u = x_1 - x_0$

Sample $t \sim Uniform([0,1])$, train for $v_\theta(x_t, t) \approx u$

Loss:

$$\mathcal{L}(\theta) = \|v_\theta(x_t, t) - u\|^2$$

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Step 2: train a conditional flow-matching model

A curve is $\dot{x} = v_\theta\left(x(t), t, \frac{n}{128}, J(x(1))\right)$. Where do we get v_θ?!

Sample a random x_0 and $x_1 = x_{elite}$ from the elite set.

Let the trajectory be $x_t = (1-t)x_0 + tx_1 \Rightarrow$ the velocity is $u = x_1 - x_0$

Sample $t \sim Uniform([0,1])$, train for $v_\theta\left(x_t, t, \frac{n}{128}, J(x_{elite})\right) \approx u$

Loss:

$$\mathcal{L}(\theta) = \left\| v_\theta\left(x_t, t, \frac{n}{128}, J(x_{elite})\right) - u \right\|^2$$

At inference: take $J_{elite} = \min J(x_{elite})$ for $J(x_{elite})$ in the elite set. Maybe this is supposed to push things further into the crème de la crème…

Step 2: train a conditional flow-matching model

A curve is $\dot{x} = v_\theta(x(t), t, c)$. Where do we get v_θ?!

Sample a random x_0 and $x_1 = x_{elite}$ from the elite set.

Let the trajectory be $x_t = (1-t)x_0 + tx_1 \Rightarrow$ the velocity is $u = x_1 - x_0$

Sample $t \sim Uniform([0,1])$, train for $v_\theta(x_t, t, c) \approx u$

Loss:

$$\mathcal{L}(\theta) = \|v_\theta(x_t, t, c) - u\|^2$$

c is conditional information

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Step 2: Geometrically-Aware Sampling (GAS)

A curve is $\dot{x} = v_\theta(x(t), t, c)$.

We sample $x(0)$ uniformly and want to get $x(1)$.

For that you, well, integrate the flow. Say, using Euler method.

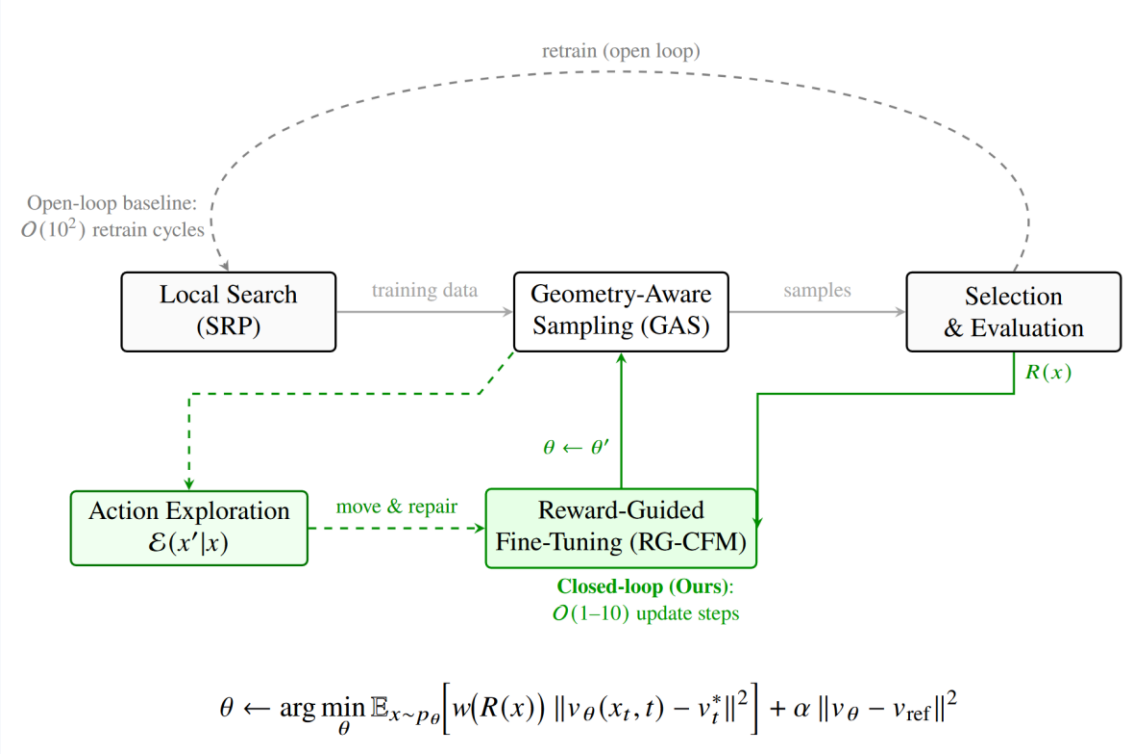
Choose Δt and do:

- $\tilde{x}_{r+1} = x_r + \Delta t \cdot v_\theta(x_r, t_r)$ (Euler step)
- $\tilde{\tilde{x}}_{r+1} = \Pi_{[0,1]^{2n}}(\tilde{x}_{r+1})$ (Clamp back to the square)
- $x_{r+1} = \Pi_{[0,1]^{2n}}\left(\tilde{\tilde{x}}_{r+1} - \eta \nabla_x \mathcal{L}_{SRP}\right)$ (Repair, several times)

There are some more steps not worth mentioning

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

The (ridiculously complicated) pipeline



Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Step 3: evaluate candidates & weight rewards

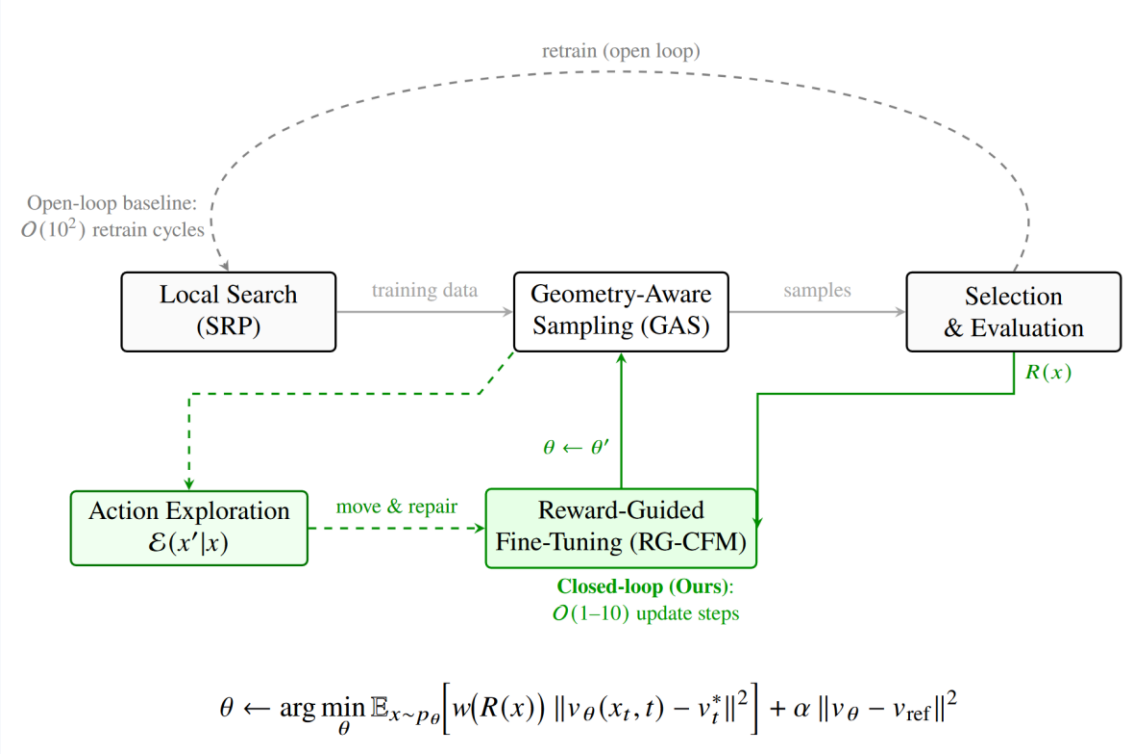
We have $\hat{x}_1, \ldots, \hat{x}_B$ sampled on the previous stage. Compute $r_b = J(\hat{x}_b)$, where

$$J(x) = \min_{1 \le i < j < k \le n} A_{ijk}$$

New elites go back to (2) to update v_θ

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

The (ridiculously complicated) pipeline



Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

Step 4: Reward-Guided Fine Tuning

Take current elites $x_1, \dots, x_B$ and perturb them towards better J_{SRP} (additional exploration)

$$x'_b = \Pi_{[0,1]^{2n}}\left(x + \rho \cdot \frac{\nabla_x J_{SRP}}{\|\nabla_x J_{SRP}\| + \epsilon}\right)$$

Normalize the rewards $r_1, \dots, r_B$ of $x'_1, \dots, x'_n$ (mean, std) $\rightarrow a_1, \dots, a_b$

Set weights for every object: $(w_1, \dots, w_B) = \mathrm{softmax}(\tau a_1, \dots, \tau a_B)$

Train v_θ on

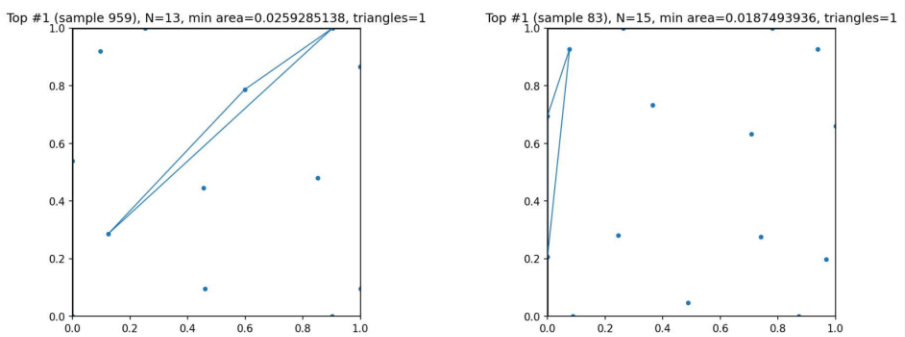
The same loss as before, just with weights

Don't stray too far from the previous state of v_θ

$$\mathcal{L}_{RG} = \frac{1}{B}\sum_{b=1}^{B} w_b \left\| v_\theta(x_{b,t}, t) - u_{b,t} \right\|^2 + \alpha \left\| v_\theta - v_{ref} \right\|^2$$

Flow-based Extremal Mathematical Structure Discovery https://arxiv.org/abs/2601.18005

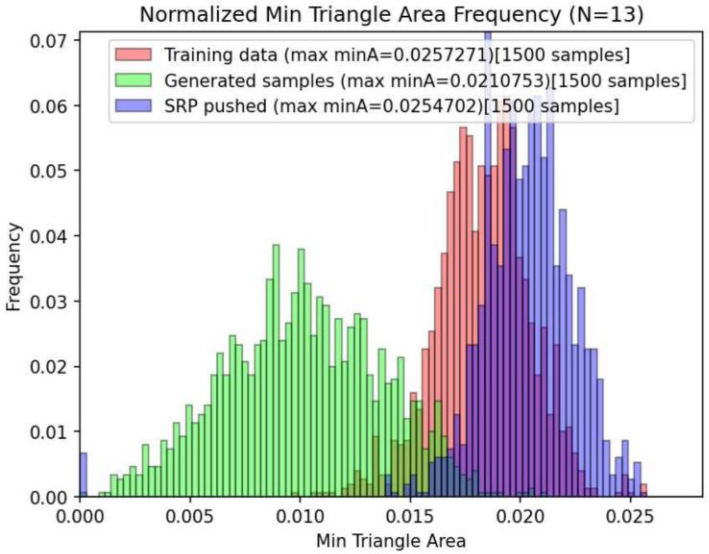
Results

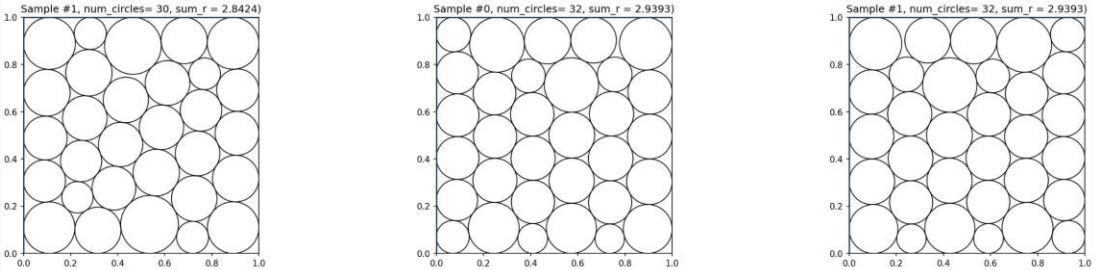


$n = 13$, $A_{\min} = 0.0259285138$ $n = 15$, $A_{\min} = 0.0187493936$

FIGURE 10. **Best Heilbronn configurations found by FLOWBOOST (after final push).** The highlighted triangle(s) attain the minimum area. The best known constructions are $A_{\min} = 0.0270$ for $n = 13$ and $A_{\min} = 0.0211$ for $n = 15$ in [65].

Maximizing sum of radii





$n = 30$, $\sum r_i = 2.8424$ (B) $n = 32$, $\sum r_i = 2.9393$ (A) $n = 32$, $\sum r_i = 2.9393$ (B)

FIGURE 14. **Best sum-of-radii circle packings found by FLOWBOOST (after final push).** For each $n \in \{26, 30, 32\}$ we show two distinct arrangements (A/B) attaining the same best objective value $\sum_{i=1}^{n} r_i$. For $n = 26$ and $n = 32$ these beat the best known result found by AlphaEvolve [2].

A further paper

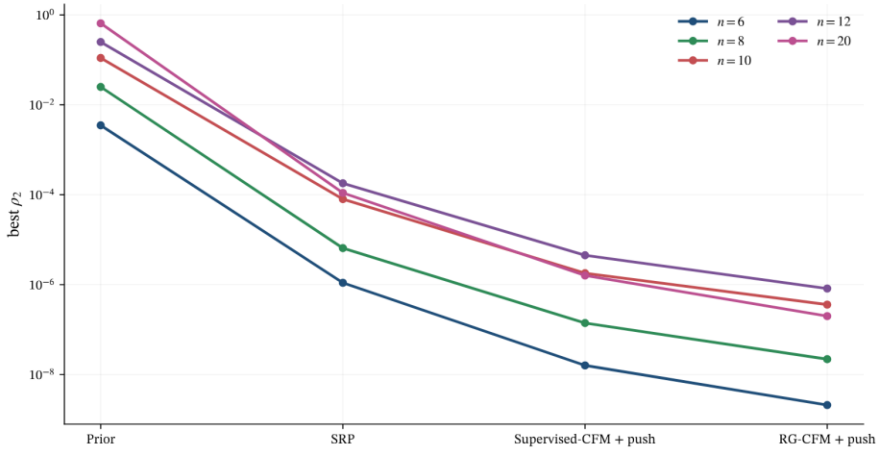


FIGURE 8. Stage-wise results for the $p = 2$ FlowBoost pipeline at representative degrees $n = 6, 8, 10, 12, 20$. SRP first identifies a high-quality feasible region, supervised CFM learns a proposal distribution from that region, and reward-guided fine-tuning sharpens the proposal distribution further so that the subsequent final push attains the best values.

Theorem 1.1 (Finite free Stam inequality [4, Theorem 1.4]). *For all monic real-rooted f, g of degree n,*

$$\frac{1}{\Phi_n(f \boxplus_n g)} \geq \frac{1}{\Phi_n(f)} + \frac{1}{\Phi_n(g)}. \tag{5}$$

Equality holds when f and g are Hermite polynomials.

Spectral Structure in Finite Free Information Inequalities and p-Stam Phase Transitions
https://arxiv.org/abs/2604.11922